

INSTITUTO SUPERIOR DE ENGENHARIA DO PORTO

Pós-Graduação em Sistemas Computacionais Embebidos  
Sistemas Operativos de Tempo-Real (PGSCE-SIOPTR)

---

SIOPTR: TT5: Week 3

# My Own KERnel (MOKER)

## version: Framework Setup

---

Paulo Baltarejo Sousa  
pbs@isep.ipp.pt  
2025/26

# 1 Introduction

---

The goal of the MOKER framework is to create a set of tools that help MOKERs students to explore and to learn the how to develop code for Linux kernel.

## 2 Preparing

---

One of the goals is to avoid spreading out files for many directories in the Linux kernel source code tree. So, it will be created a directory, called `moker` in the Linux kernel source code tree to encompass all files required for such a framework.

For that purpose, open a terminal `$HOME/linux-kernel` directory:  
and type:

```
cd linux-6.19.9-moker
cd kernel
mkdiroker
```

## 3 Configuration Option Menu

---

The next step is to add a new configuration option entry that is used to wrap all the MOKER framework code.

Then, the `Kconfig` file has to be created in the `linux-6.19.9-moker/kernel/moker` directory.

The content of the `Kconfig` file is:

```
menu "MOKER framework"
config MOKER_FRAMEWORK
bool "My Own KERnel Framework"
default y
endmenu
```

In spite of the entry name is `MOKER_FRAMEWORK`, in the code it will be referred as `CONFIG_MOKER_FRAMEWORK`. The `CONFIG_` prefix is assumed but is not written. The `bool` directive states that this option entry is a feature and can only be set using two values (y or n). The quoted text following the directive provides the name of this option in the various configuration utilities, like `make menuconfig`. The default value of this option is defined using `default` directive.

Since the host system used in this document is based on a x86 architecture, the `linux-6.19.9-moker/kernel/moker/Kconfig` configuration file must be included into the root configuration file for x86 architecture located at `linux-6.19.9-moker/arch/x86/Kconfig` file.

Therefore, update the `linux-6.19.9-moker/arch/x86/Kconfig` file adding (at the end of file) a new entry `source "kernel/moker/Kconfig"`:

```
...

config HAVE_ATOMIC_IOMAP
def_bool y
depends on X86_32

source "arch/x86/kvm/Kconfig"

source "arch/x86/Kconfig.cpufeatures"

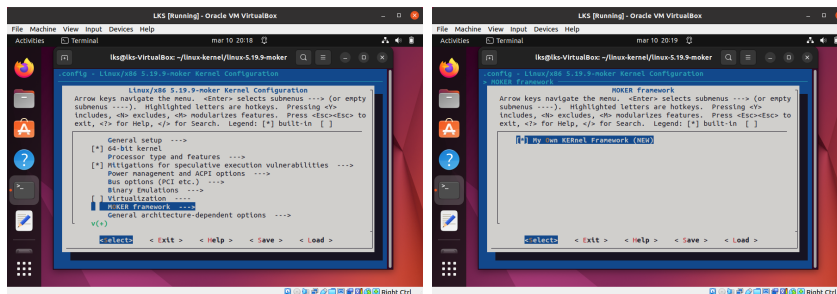
source "arch/x86/Kconfig.assembler"

source "arch/x86/Kconfig.assembler"

source "kernel/moker/Kconfig"
```

Open a terminal, move to the `linux-6.19.9-moker` directory and type:  
`make menuconfig`

Use the "Down" arrow key to move to the last option, called "MOKER Framework" that you have just created.



## 4 Compilation

As mentioned before, all MOKER framework implementation source code files will be placed into the `linux-6.19.9-moker/kernel/moker` directory.

The next step is to create a Makefile file for compiling the MOKER Framework source code.

Create an empty file called `Makefile` in the `linux-6.19.9-moker/kernel/moker` directory and add a comment line.

```
# MOKER framework makefile
```

Next, update the `kernel` directory `Makefile` file (located at `linux-6.19.9-moker/kernel/Makefile`) file to include `moker` directory in the compilation process:

```
...
obj-$(CONFIG_HAS_IOMEM) += iomem.o
obj-$(CONFIG_RSEQ) += rseq.o
obj-$(CONFIG_WATCH_QUEUE) += watch_queue.o

obj-$(CONFIG_RESOURCE_KUNIT_TEST) += resource_kunit.o
obj-$(CONFIG_SYSCTL_KUNIT_TEST) += sysctl-test.o

#include MOKER framework
obj-$(CONFIG_MOKER_FRAMEWORK) += menger/

CFLAGS_kstack_erase.o += $(DISABLE_KSTACK_ERASE)
CFLAGS_kstack_erase.o += $(call cc-option,-mgeneral-regs-only)
obj-$(CONFIG_KSTACK_ERASE) += kstack_erase.o
KASAN_SANITIZE_kstack_erase.o := n
KCSAN_SANITIZE_kstack_erase.o := n
KCOV_INSTRUMENT_kstack_erase.o := n
...
```

Now, you are ready to compile the Linux kernel. Open a terminal `$HOME/linux-kernel` directory:

1. Create a new `.config` file with the new configuration option (`MOKER_FRAMEWORK`)  
`cd linux-6.19.9-moker`  
`make menuconfig`  
Select "Exit" and "Yes" for saving the new `.config` file.  
`cd ..`
2. Compile and install the kernel:  
`sudo ./kcompile.sh`  
`sudo reboot`